

HW#3 Due Tue. 11/12 Midnight (same rules as usual)

- In Collab: upload your code (only original code will be evaluated. Do not use any downloaded toolboxes from the web) and a brief instruction on how to run it, together with a pdf document containing the data and plots requested below. Please upload a zip file containing all of these files. The file name should contain your family_name and first_name (e.g. Bezzo_Nicola_HW3.zip). HW received after the deadline will receive 1/2 points.
- 1. **Kalman Filter (50pt):** Create a 100mx100m environment. Simulate a go-to-goal behavior with constant cruise control speed between two points, 50m apart. Implement a simple point mass robot. The robot has 2 GPS sensors (s_1 and s_2) estimating its position (i.e., each sensor return x-y position of the robot). The sensor errors are modeled as 0 mean Gaussian distributions
 - For s_1 $\mu_{1x} = \mu_{1y} = N(0, 6)$
 - For s_2 $\mu_{2x} = \mu_{2y} = N(0, 4)$
 - The input u to the robot has noise modeled as 0 mean Gaussian $v = N(0, 1)$
- Implement a Kalman Filter to estimate the state of the system as it is running to the goal. Estimate both the position and the velocity of the robot.
- Plot estimated robot position (x,y), linear velocity, and distance to the goal over time.
- On another figure plot each sensor measurement together with the estimate from the filter. You should see that the estimate is better than the measurements

2. **Particle Filter (50pt):** Create a 100mx100m environment. Create 6 landmarks at the following locations $\{(25,25)(25,70)(70,25)(70,70)(10,40)(80,60)\}$
- Place your robot in a random position and orientation in the middle of the environment
 - Guess an initial pose of your robot in the environment
 - Create 1000 particles scattered uniformly through the environment. Each particle should contain (random) x,y position and orientation.
 - The robot can measure the distance to each landmark with some error $\mu = N(0, 8)$.
 - The robot forward and turning motion noise is $v = N(0, 0.5)$
 - Note that these errors apply also to the particles!
 - Implement a particle filter to localize the robot after 10 steps. In particular pay attention on implementing
 - a weight function as shown in class
 - the resampling procedure as shown in class
 - NOTE: at each step make the robot move forward and rotate. It's up to you to decide the control input but the robot should not jump around. You can try with "move forward 1m" and "rotate 0.2 rad" at every step.
 - Print the 10 estimated poses of the robot and the error between the actual pose and each estimate. HINT: average all particle values.
 - **EXTRA (10pt):** re-implement everything considering that your sensor has limited range = 30m